Sentence embedding

Tìm câu gần nghĩa và đọc thứ hạng

Tìm câu cùng ý khi chữ khác nhau

Query: “quên thông tin đăng nhập”

Tài liệu: “đặt lại mật khẩu”

Tìm theo chữ: hai bên không chung từ nào.

Muốn tìm theo ý: cần một không gian vector được học phù hợp.

Mục tiêu

- Tính cosine similarity; tìm kiếm bằng tích vô hướng sau khi chuẩn hóa.
- Gộp vector token thành vector câu, bỏ qua PAD.
- Chấm hệ tìm kiếm bằng Recall@k và MRR.
- So TF-IDF với các loại sentence embedding.

01

Câu thành vector

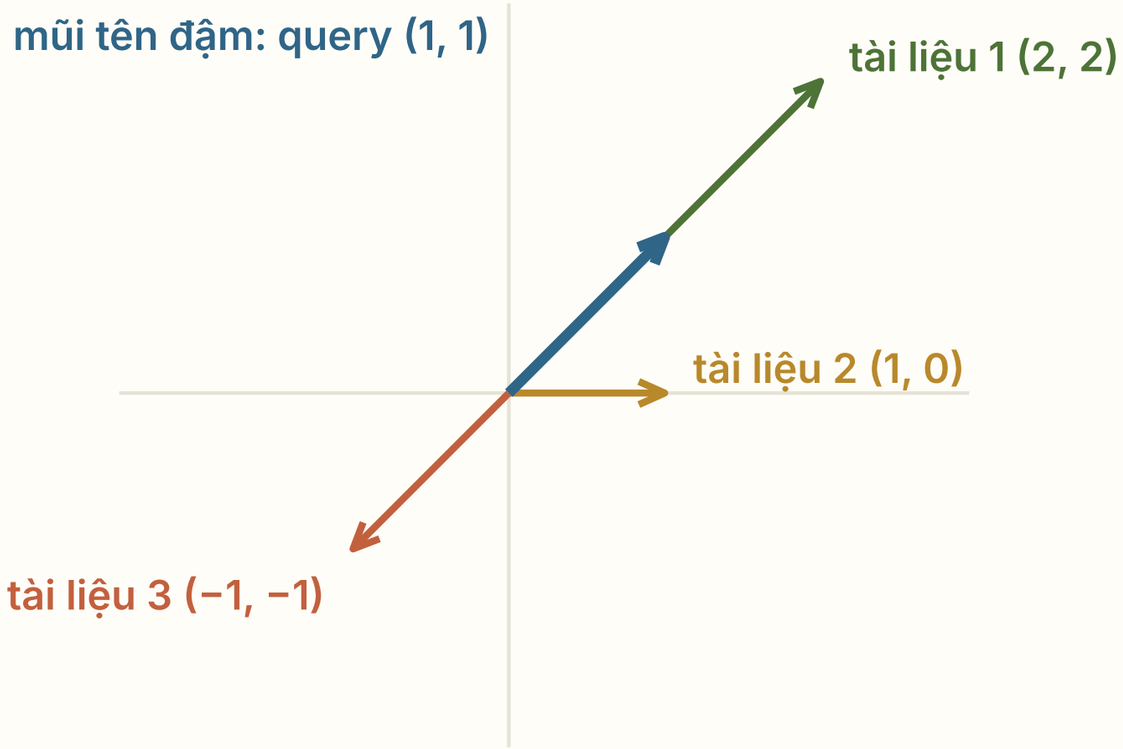
Embedding phụ thuộc mục tiêu học

Một vector cho cả câu.

Ví dụ: 4 câu với cho ma trận shape `(4, 384)`.

Từng chiều không nhất thiết có tên. Trung bình hidden state của model chỉ học điền từ chưa chắc tốt cho tìm kiếm.

Cosine nhìn hướng



cosine với query (1, 1)

tài liệu 1	1
tài liệu 2	0.707
tài liệu 3	−1

cosine chỉ nhìn hướng, không nhìn độ dài

Cùng hướng cho cosine 1 dù dài gấp đôi; lệch 45° cho 0.707; ngược hướng cho −1.

Cosine similarity

Sau chuẩn hóa L2, cosine chính là tích vô hướng.

0.8 là một điểm tương tự, chưa phải xác suất 80%.

Ba tài liệu, một query

KẾT QUẢ

```
[[ 1.     0.707 -1.   ]]
```

Vector 0 không có góc xác định.

Sau chuẩn hóa: `D @ q` cho điểm.

HỎI NHANH

Hai vector và có cosine bằng bao nhiêu?

A 15

B 3

C 1

D 1/3

HỎI NHANH

Query là . Tài liệu A là , tài liệu B là . Xếp theo tích vô hướng thô thì tài liệu nào đứng đầu? Xếp theo cosine thì sao?

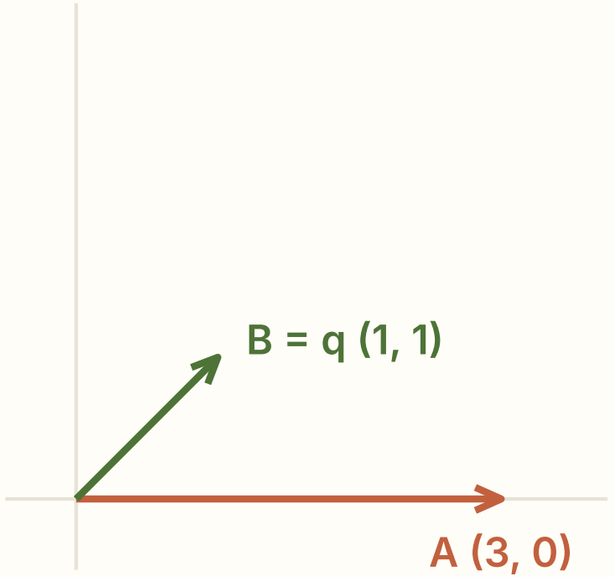
A Theo tích vô hướng là A, theo cosine là B

B Theo tích vô hướng là B, theo cosine là B

C Theo tích vô hướng là A, theo cosine là A

D Theo tích vô hướng là B, theo cosine là A

Độ dài đổi thứ hạng



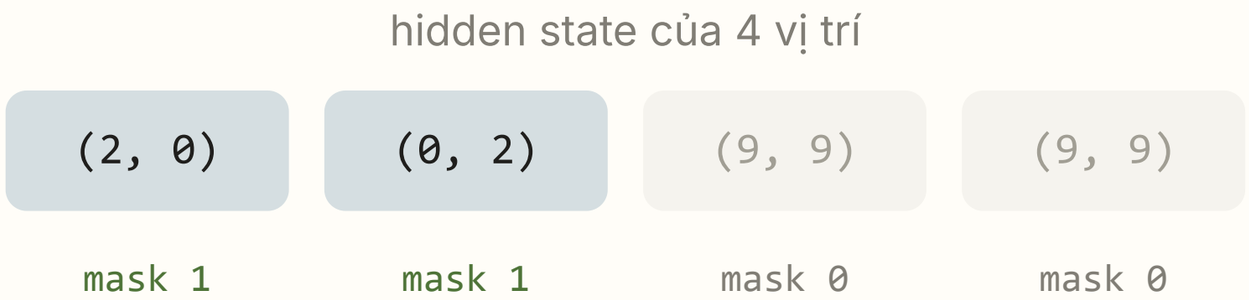
	tích vô hướng	cosine
A	3	0.707
B	2	1
	A thắng: dài	B thắng: cùng hướng

Tích vô hướng ưu ái vector dài; cosine chỉ xét hướng.

02

Từ token về câu

Padding không góp mean



$$\Sigma\ m_i\ h_i\ /\ \Sigma\ m_i = ((2, 0) + (0, 2)) / 2$$

padding không được giả định bằng 0: hidden state ở đó có thể là bất cứ gì

Có mask ra (1, 1); lấy trung bình cả PAD bị kéo về (5, 5).

Pooling đúng shape

```
def masked_mean(hidden_states, mask):
    weight = mask.unsqueeze(-1)
    weight = weight.to(hidden_states.dtype)
    token_count = weight.sum(dim=1).clamp_min(1)
    weighted_sum = (
        hidden_states * weight
    ).sum(dim=1)
    return weighted_sum / token_count
```

H: `(B, L, d)`

`mask`: `(B, L)` → `(B, L, 1)`

Kết quả: `(B, d)`

Câu rỗng phải được xử lý riêng.

HỎI NHANH

Encoder đã ở `eval()` và nhận đủ `input_ids` lẫn `attention_mask`. Encode câu “mượn sách” một mình thì được vector . Encode nó chung batch với một câu dài hơn thì được , và . Nguyên nhân khả dĩ nhất là gì?

A Dropout vẫn chạy, nên hai lần cho kết quả khác nhau

B Pooling lấy trung bình cả các vị trí padding

C float32 làm tròn khác nhau giữa hai batch

D Tokenizer tách “mượn sách” khác đi tùy theo batch

HỎI NHANH

Sau masked mean, pipeline luôn chuẩn hóa L2 rồi mới tính cosine. Trong `masked_mean`, một đội lỡ viết `token_count = weight.sum()`, tức tổng số token thật của cả batch, thay vì của từng câu. Thứ hạng retrieval thay đổi thế nào?

A Câu ngắn bị kéo về phía các vị trí padding

B Code báo lỗi shape, vì mẫu số chỉ còn một số vô hướng

C Mọi score cosine giảm theo số câu trong batch

D Thứ hạng không đổi, vì hướng của mỗi vector giữ nguyên

03

Thí nghiệm retrieval nhỏ

Mốc TF-IDF

Tài liệu: mật khẩu, sách, thời khóa biểu.

Query: “quên truy_cập”, “mượn sách”, “lịch học”.

KẾT QUẢ

```
[[0. 0. 0.]
 [0. 1. 0.]
 [0. 0. 0.]]
```

Hàng toàn 0 không cho bằng chứng chọn top-1.

HỎI NHANH

Thêm tài liệu thứ tư “đăng_ký học thêm” vào `documents`, rồi fit lại TF-IDF. Bây giờ query “lịch học” cho kết quả gì?

A Vẫn toàn 0, vì “lịch” không có trong vocabulary

B Tài liệu mới đứng đầu với 0.577, còn gold vẫn được 0

C Gold lên đầu, vì “học” gần nghĩa với “thời_khóa_biểu”

D Bốn tài liệu bằng điểm nhau, vì IDF chia đều

Embedding học từ nhãn chủ đề

9 câu train, 3 chủ đề.

Masked mean của token, kéo về mục tiêu one-hot bằng MSE.

Mục tiêu shape `(9, 3)`.

KẾT QUẢ

```
MSE đầu/cuối: 0.7867 0.0
```

MSE 0 đo train, chưa đo retrieval.

Không attention, không ngữ cảnh.

Tìm trên cùng bảng embedding

KẾT QUẢ

```
[[ 0.845  0.211 -0.491]
 [-0.     1.    -0.   ]
 [-0.     0.     1.   ]]
top-1: [0, 1, 2]
```

“quên truy_cập”: câu mới, 0.845 với mật khẩu.

Hai hàng sau: query chính là câu train.

Mean token không giữ thứ tự.

Hai ma trận điểm

TF-IDF

	mật khẩu	mượn sách	thời khóa biểu
quên truy_cập	0.00	0.00	0.00
mượn sách	0.00	1.00	0.00
lịch học	0.00	0.00	0.00

embedding nhỏ

	mật khẩu	mượn sách	thời khóa biểu
quên truy_cập	0.84	0.21	−0.49
mượn sách	−0.00	1.00	−0.00
lịch học	−0.00	0.00	1.00

hàng: query; cột: tài liệu; đường chéo là gold

TF-IDF chỉ khớp được câu trùng từ; embedding tìm đúng cả câu khác từ.

MSE chỉ ràng buộc trung bình câu

KẾT QUẢ

```
quên: [ 0.95  0.41 -0.66]
mật_khẩu: [ 1.05 -0.41  0.66]
trung bình: [1. 0. 0.]
[[ 0.725 -0.272  0.633]
 [ 0.357  0.901 -0.248]]
```

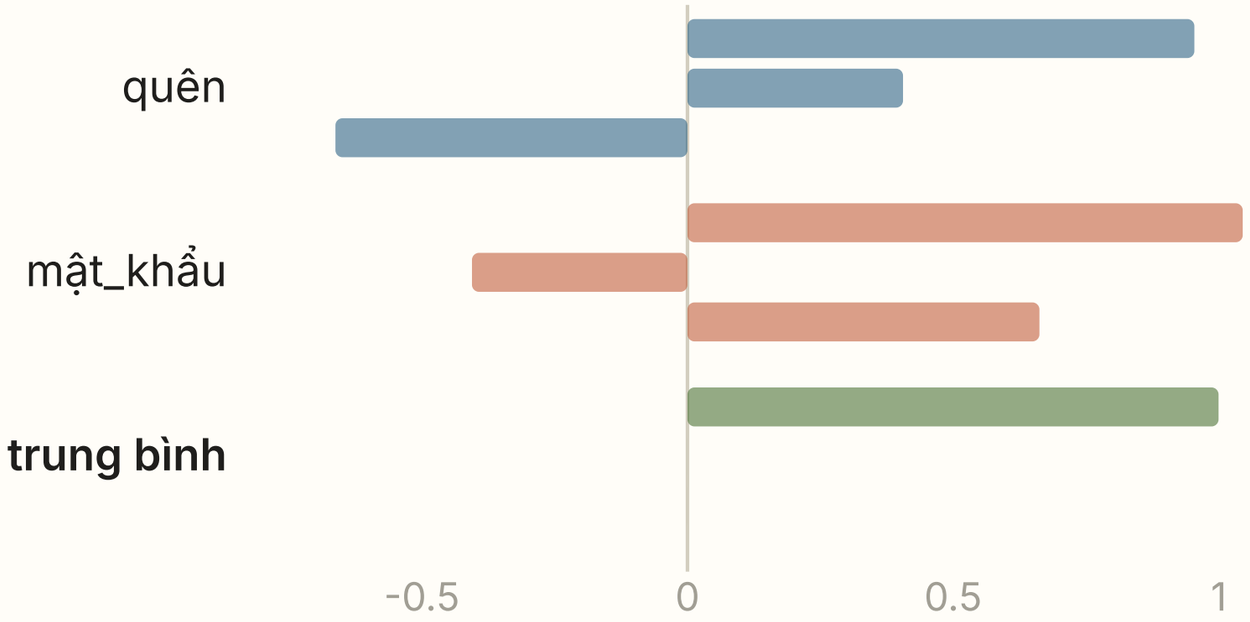
Phần dư triệt tiêu trong câu train.

“khôi_phục mật_khẩu”: 0.725 so với 0.633, suýt thua.

“quên thẻ thư_viện” nghiêng về thư viện.

Seed 0: cả hai query ghép chọn sai.

Phần dư của từng từ



ba chiều: tài khoản, thư viện, lịch học

phần dư triệt tiêu trong câu train

Hai vector từ đối dấu nhau ở hai chiều nên trung bình rơi đúng (1, 0, 0).

HỎI NHANH

Trong train, `đặt_lại` cũng chỉ xuất hiện trong một câu, cùng với `mật_khẩu`, với mục tiêu . Khi MSE về gần 0, vector của `đặt_lại` gần với vector nào nhất?

A Vector , đúng mục tiêu của chủ đề

B Vector của `mật_khẩu`, từ đi cùng nó trong câu

C Vector của `quên`, khoảng

D Không đoán được, vì mỗi hàng giữ khởi tạo riêng

Metric thứ hạng

Recall@k: gold nằm trong top ở bao nhiêu query?

Hai query ở hạng 1 và 2: MRR .

Đọc cả điểm và hạng

TF-IDF: rank `[1,1,3]`, R@1 0.667, MRR 0.778.

Embedding nhỏ: rank `[1,1,1]`, R@1 1.0, MRR 1.0.

TF-IDF query đầu đúng do phá hòa.

Embedding nhỏ: hai query là câu train.

Thứ hạng từng query

	quên truy_cập	mượn sách	lịch học	R@1	MRR
TF-IDF	hạng 1	hạng 1	hạng 3	0.667	0.778
embedding nhỏ	hạng 1	hạng 1	hạng 1	1.0	1.0

hạng 1 của TF-IDF ở “quên truy_cập” chỉ nhờ phá hòa khi cả hàng bằng 0

Hạng của tài liệu đúng: TF-IDF 1, 1, 3; embedding 1 cả ba.

HỎI NHANH

Giữ nguyên query và điểm TF-IDF, chỉ xếp lại kho thành [“mượn sách”, “xem thời_khóa_biểu”, “đặt_lại mật_khẩu”]. R@1 và MRR của TF-IDF thành bao nhiêu?

A R@1 khoảng 0.333, MRR khoảng 0.611

B R@1 khoảng 0.667, MRR khoảng 0.778

C R@1 khoảng 0.333, MRR khoảng 0.778

D R@1 khoảng 0.667, MRR khoảng 0.611

Validation cần gì

- Query do người khác viết, không chép từ train.
- Hard negatives: chung chữ, khác việc.
- Đề chấm trên FAQ mới: mọi cách hỏi của một FAQ nằm cùng một phía.

04

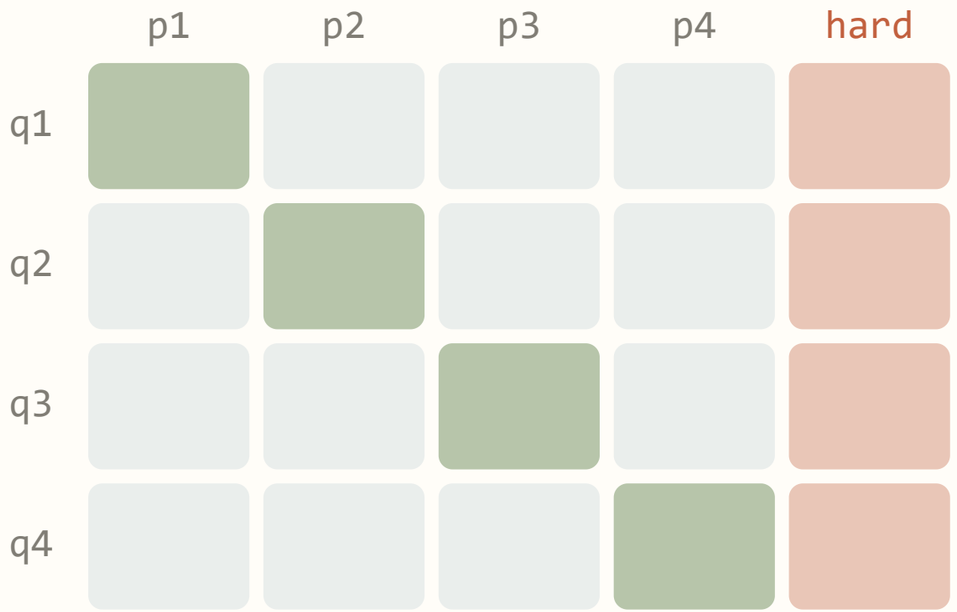
Model câu và ứng dụng

Học từ cặp câu

Hàng : phân loại lớp, lớp đúng là .

cột còn lại: in-batch negatives, như CLIP và SimCLR ở bài 5.7.

Ma trận của một batch



đường chéo: cặp đúng

ô khác: in-batch negative

cột thêm: hard negative

mỗi hàng là softmax B + 1 lớp

Mỗi query phải chấm tài liệu của cặp mình cao hơn tài liệu của các cặp khác trong batch.

Negative khó, nhưng phải sai thật

- Negative ngẫu nhiên: quá dễ, gần như không có gradient.
- Hard negative: chung chữ hoặc chủ đề, không trả lời query.
- Bản diễn đạt lại của gold bị coi là negative: false negative.
- Hai cặp chung tài liệu đúng trong một batch: cùng lỗi.

HỎI NHANH

Khi train bằng contrastive loss, mỗi query được thêm một hard negative. Query “đổi mật khẩu” có tài liệu đúng là “Hướng dẫn đặt lại mật khẩu”. Nên chọn hard negative nào?

A “Cách mượn sách thư viện”, khác hẳn chủ đề

B Một tài liệu ngẫu nhiên khác trong kho

C “Các bước đổi mật khẩu tài khoản”, chung nhiều chữ nhất

D “Hướng dẫn đổi lịch học”, chung chữ “đổi” nhưng khác việc

Chạy với model pretrained

- `paraphrase-multilingual-MiniLM-L12-v2`, bản tải sẵn trên máy.
- Kiểm `1_Pooling/config.json`: mean pooling.
- Dừng nếu câu vượt `max_seq_length`.
- Cùng 3 tài liệu, 3 query: ma trận và top-1.

Đọc hàng “Tôi quên thông tin đăng nhập” và khoảng cách top-1 với top-2.

Kho lớn

- Encode kho một lần; cache kèm model, revision, pooling.
- Tài liệu dài: chia đoạn có chồng lấn, gộp điểm.
- Brute force: số `float32`, khoảng 154 MB.
- Ngưỡng và chọn bằng validation.

HỎI NHANH

Tài liệu nội quy thư viện dài 40 đoạn, mỗi đoạn được encode riêng. Chỉ đoạn 17 trả lời query “mất thẻ thì làm gì”. Nên gộp điểm của cả tài liệu thế nào?

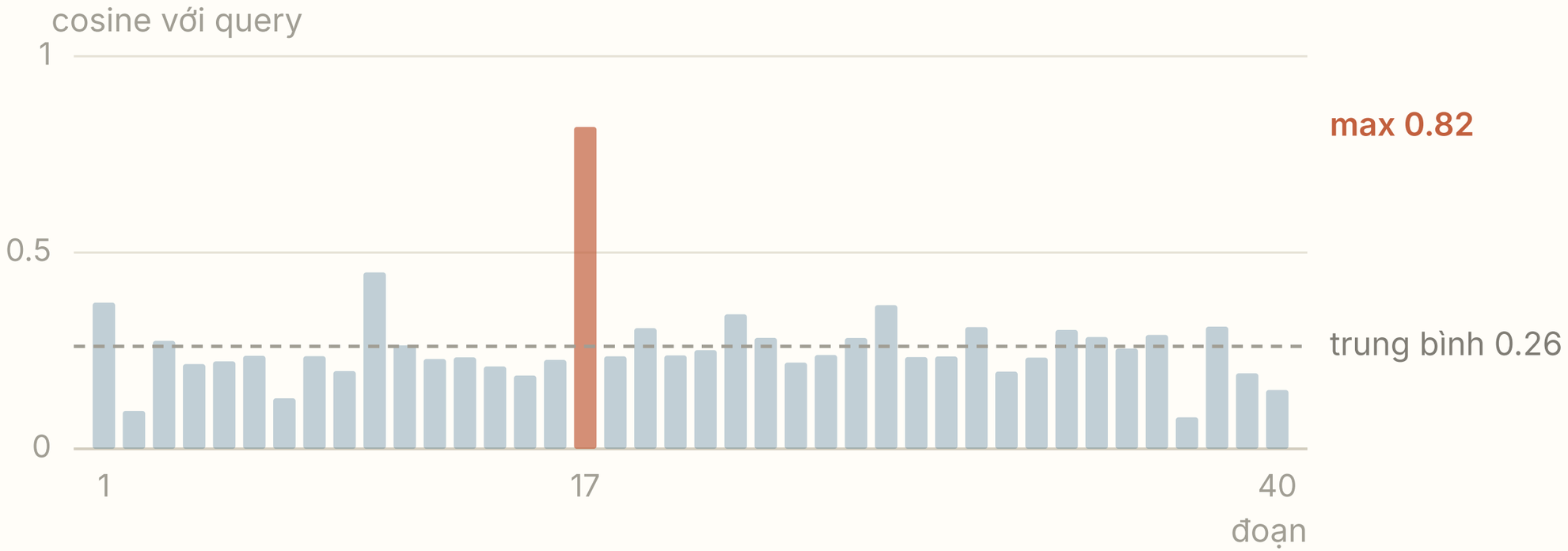
A Lấy trung bình điểm của 40 đoạn cho công bằng

B Lấy điểm lớn nhất trong 40 đoạn

C Chỉ encode đoạn đầu, vì đoạn đầu thường tóm tắt

D Nối 40 đoạn lại, để tokenizer tự cắt

Max hay trung bình

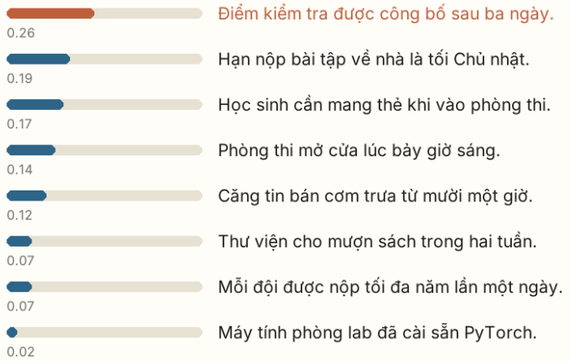


Lấy điểm lớn nhất giữ được đoạn trả lời; lấy trung bình bị 39 đoạn khác pha loãng.

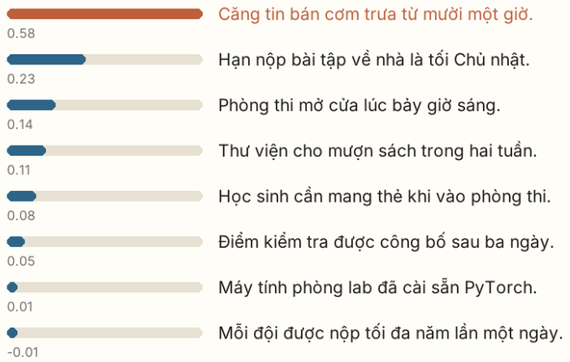
Tìm câu trả lời theo nghĩa



câu hỏi: “Mấy giờ thì được vào phòng thi?”



câu hỏi: “Khi nào biết điểm?”



câu hỏi: “Ăn trưa ở đâu?”



câu hỏi: “Máy ở phòng thực hành có thư viện học sâu không?”

Model câu đa ngôn ngữ tìm câu trả lời theo cosine, dù gần như không trùng từ.

Phản hồi gần nghĩa

- Giữ tập ứng viên và lịch sử đoán.
- Lọc: giữ ứng viên có khớp score trong sai số.
- Đoán từ tách ứng viên còn lại nhiều nhất.
- Gần nghĩa không phải quan hệ logic (NLI).

HỎI NHANH

Còn ba ứng viên A, B, C cho từ bí mật. Score của judge lệch tối đa so với cosine tính theo embedding công khai. Mỗi phương án dưới đây là một từ có thể đoán, kèm cosine dự đoán với A, B và C. Đoán từ nào thì chỉ một lần phản hồi là chắc chắn còn lại đúng một ứng viên?

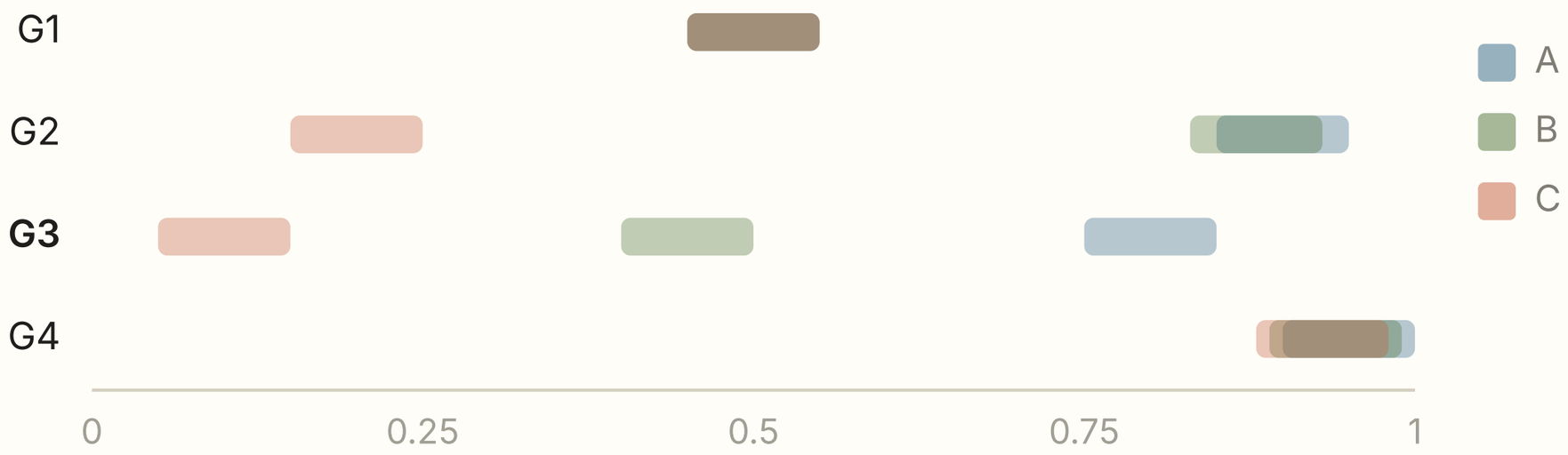
A G1, với 0.50, 0.50, 0.50

B G2, với 0.90, 0.88, 0.20

C G3, với 0.80, 0.45, 0.10

D G4, với 0.95, 0.94, 0.93

Khoảng nào tách được ứng viên



mỗi thanh: cosine dự đoán ± 0.05; chỉ G3 có ba khoảng rời nhau

Chỉ với G3, ba khoảng cosine rời nhau.

Khi đi thi

- Mốc TF-IDF trước embedding.
- Hàm chấm giống hệt đề.
- Đủ ID gốc, không trùng, đúng thứ tự.

05

Luyện đề

LUYỆN ĐỀ 1

`v` có shape `(N, 100)`. Không dùng `np.linalg`, dòng nào chuẩn hóa từng hàng về độ dài 1?

A `v / np.sqrt((v**2).sum(1))`

B `v / np.sqrt((v**2).sum(0, keepdims=True))`

C `v / np.sqrt((v**2).sum(1, keepdims=True))`

D `v / (v**2).sum(1, keepdims=True)`

PHỎNG THEO USA-NA-AIO 2026 · Vòng 1 · Part 5.4

LUYỆN ĐỀ 2

Đội train embedding bằng contrastive loss với nhiệt độ , còn đề chấm bằng MRR@10. Cần chọn trong bằng 5-fold cross-validation. Với mỗi , làm theo quy trình nào?

A Bỏ loss, tối ưu thẳng MRR@10 bằng gradient trên 4 fold

B Train bằng loss( ) trên 4 fold, rồi đo MRR@10 ở fold còn lại

C Train bằng loss( ) trên toàn bộ, chọn có loss train thấp nhất

D Train bằng loss( ) trên 4 fold, chọn có loss ở fold còn lại thấp nhất

PHỎNG THEO MIT 6.036 · Final, Spring 2022 · Q5(f)

LUYỆN ĐỀ 3

Sáng nay trang hỏi đáp thêm 20 bài hướng dẫn mới, chưa ai mở. Hệ (1) gợi ý theo lịch sử bấm của người dùng, tức collaborative filtering. Hệ (2) gợi ý theo embedding nội dung bài, tức content filtering. Hệ nào đưa được bài mới vào gợi ý ngay?

A Chỉ hệ (2), vì nó dùng nội dung của bài

B Chỉ hệ (1), vì người dùng sẽ bấm thử bài mới

C Cả hai, vì đều xếp hạng bằng tích vô hướng

D Không hệ nào, cho đến khi có đủ lượt bấm

PHỎNG THEO CMU 10-601 · Exam 3 Practice · Q7.6

Chốt ý

Cosine đo hướng; mask quyết định token nào vào mean.

Model câu học từ cặp câu; retrieval cần gold, thứ hạng và query độc lập.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; thêm một query ghép từ hai nhóm, đoán trước top-1 rồi kiểm lại.